



Unlock 20+ free resources — guides, roadmaps & career resources for the TBE community.

[Get free access →](#)

[↑ Share](#)

[Deep Read](#)



2026 EDITION

— THE COMPLETE PATH — AUTONOMOUS AI SYSTEMS

Agentic AI Engineer Roadmap

From Python & LLM basics to building fully autonomous AI agents in 2026. Orchestration, tool use, memory systems, multi-agent pipelines, evals — everything you need with free YouTube resources for every phase.



"The next wave isn't AI that answers questions. It's AI that acts, plans, and executes — end to end, without a human in the loop. The engineer who can build that is the most sought-after person in tech right now."

— The Boring Education Team

12-18

MONTHS TO JOB-READY

11

PHASES TO MASTER

50+

FREE YT RESOURCES

∞

CAREER CEILING

// FOUNDATION LAYER

Start Here — Python, LLMs & APIs

1

• Weeks 1–4

PHASE 01 • PYTHON FOR AGENTS

Python Essentials for Agentic Development

Agents are primarily written in Python. You need more than syntax — you need fluency in **async** programming, API design, and the ecosystem. Master **async/await** for non-blocking agent loops. Understand **type hints**, **Pydantic models** and data validation — LLM responses need strict schemas. Learn **environment management** with **dotenv** and secrets handling. Practice **REST API consumption** with **httpx** and **requests**. Understand JSON schema, function signatures, and dict manipulation — these are how agents communicate with tools.

Non-negotiable

Python async/await

Pydantic v2

Type hints

httpx / requests

JSON schema

dotenv / secrets

venv / uv

▶ Python Full Course - freeCodeCamp

▶ Python Async/Await - Tech With Tim

▶ Pydantic v2 Crash Course - ArjanCodes

▶ REST APIs with Python - freeCodeCamp

2

• Weeks 3–7

PHASE 02 • LLM FUNDAMENTALS

How LLMs Work — Tokens, Attention & Context Windows

You cannot build reliable agents without understanding the engine under the hood. Study **tokenization**: BPE, token limits, cost implications. Understand **attention mechanisms** and why context order matters. Learn **temperature**, **top-p**, and **sampling strategies** — agents need deterministic behavior (low temp), humans want creativity (high temp). Understand **context windows**: 8k vs 128k vs 1M — and what fits inside. Study the differences between GPT-4o, Claude 3.5, Gemini 1.5, Llama 3.1 — each has different strengths for agentic tasks. This knowledge will save you hours of debugging.

Core knowledge

Tokenization / BPE

Attention mechanism

Context windows

Temperature / top-p

GPT-4o / Claude 3.5

Llama 3.1 / Mistral

Model benchmarking

▶ Neural Networks - 3Blue1Brown

▶ Transformers from Scratch - Andrej Karpathy

▶ LLMs from Scratch - Andrej Karpathy

▶ Intro to LLMs - Andrej Karpathy

3

• Weeks 5–10

PHASE 03 • PROMPT ENGINEERING

Advanced Prompting — The Backbone of Every Agent

Prompt engineering is not just "write a good sentence." For agents, it is system design. Master **system prompts**: role definition, persona, constraints, output format. Learn **few-shot prompting**: 3–5 examples in-context dramatically improve reliability. Study **Chain-of-Thought (CoT)**: forcing step-by-step reasoning reduces hallucinations by 40–60%. Understand **ReAct pattern** (Reason + Act): the primary loop used in all production agents. Learn prompt injection defense — a critical security skill. Practice **output structuring**: JSON-mode, XML tags, and function-calling schemas. Reliable prompting = reliable agents.

Agent backbone

System prompts

Few-shot prompting

Chain-of-Thought

ReAct pattern

JSON / XML output

Prompt injection

Function calling

▶ Prompt Engineering Guide - freeCodeCamp

▶ Advanced Prompting - Andrej Karpathy

▶ Chain-of-Thought Prompting - AI Explained

▶ Function Calling Explained - Fireship

// TOOL USE & MEMORY SYSTEMS

Function Calling, RAG & Agent Memory

4

• Weeks 8–14

PHASE 04 • TOOL USE & FUNCTION CALLING

Giving Agents Hands — Tools, APIs & the Real World

An agent without tools is just a chatbot. Tool use is what makes agents *act*. Learn **OpenAI function calling** and **Anthropic tool use** — define JSON schemas, handle tool results, manage multi-turn tool loops. Build your own tools: **web search** (Tavily, Serper API), **code execution** (E2B sandboxes, Python REPL), **file I/O**, **browser automation** (Playwright, Puppeteer), **database queries**, and **external API calls**. Understand **tool selection strategy**: how to design tool schemas so LLMs reliably pick the right tool. Study **parallel tool execution** for speed. Learn error handling when tools fail — agents must retry gracefully.

What makes agents agents

Function calling

Anthropic tool use

Tavily / Serper

E2B code sandbox

Playwright

Parallel tool use

Error recovery

▶ Function Calling Deep Dive - Fireship

▶ OpenAI Tools Tutorial - Patrick Loeber

▶ Playwright Python - freeCodeCamp

▶ Tool-using Agents - LangChain

▶ E2B Code Execution - freeCodeCamp

5

• Weeks 10–18

PHASE 05 • RAG & AGENT MEMORY

Memory Architecture — Short-Term, Long-Term & Semantic Search

Agents without memory are amnesiac. Build all four memory types: **In-context memory** (conversation history in the prompt window — simple but limited), **External memory** via vector databases (Pinecone, Qdrant, ChromaDB) for semantic search over docs/chat history, **Episodic memory** (storing past task outcomes for future reference), and **Procedural memory** (system prompts encoding skills and behaviors). Master the full **RAG pipeline**: chunking strategies, embedding models (OpenAI text-embedding-3, BGE, Nomic), vector DB indexing, hybrid search (dense + sparse), and **re-ranking** with Cohere Rerank or BGE. Build a memory manager that decides what to store, retrieve, and forget.

Core architecture

Vector DBs (Qdrant)

Embedding models

Hybrid search

Chunking strategies

Re-ranking

Episodic memory

Mem0 / Zep

▶ RAG Tutorial - freeCodeCamp

▶ Vector DBs Explained - Fireship

▶ Advanced RAG Techniques - Hugging Face

▶ Hybrid Search & Reranking - James Briggs



Memory is your agent's biggest lever. The difference between a toy demo and a production agent is almost always memory design. How does your agent remember what it did last week? What it learned about the user? Use Mem0 or build a custom memory layer — but build something. Stateless agents don't survive contact with real users.

// MEMORY ARCHITECTURE COMPARISON

MEMORY TYPE	STORAGE	BEST FOR	TOOLS
In-Context	LLM context window	Short conversation history, few recent facts	Message array, summarization
Semantic (Vector)	Vector database	Docs, knowledge base, past conversations	Qdrant, Pinecone, ChromaDB

// AGENT FRAMEWORKS & ORCHESTRATION

LangChain, LangGraph, AutoGen & Beyond

6

• Weeks 14–22

PHASE 06 • AGENT FRAMEWORKS

LangChain, LangGraph, LlamaIndex & the Orchestration Layer

Frameworks abstract the hard parts so you can focus on logic. Start with **LangChain**: chains, agents, tools, memory, callbacks — it's the most popular and has the most tutorials. Then master **LangGraph** — the graph-based successor for stateful agent workflows. LangGraph models agents as directed graphs with nodes (LLM calls/tools) and edges (conditional routing). Learn **LlamaIndex** for data-heavy agents: document ingestion, query engines, sub-question decomposition, and agentic RAG. Study **LangSmith** for tracing and debugging agent runs — you cannot debug an agent without observability. Understand when to use frameworks vs. building from scratch (LangGraph for complex state, raw API calls for simple agents).

Orchestration layer

LangChain v0.3

LangGraph

LlamaIndex

LangSmith tracing

Stateful graphs

Conditional routing

Agent observability

▶ [LangChain Full Course - freeCodeCamp](#)▶ [LangGraph Tutorial - LangChain](#)▶ [LlamaIndex Full Course - freeCodeCamp](#)▶ [LangSmith Debugging - LangChain](#)▶ [Agentic RAG with LlamaIndex - AI Jason](#)

7

• Weeks 18–26

PHASE 07 • MULTI-AGENT SYSTEMS

Multi-Agent Orchestration — CrewAI, AutoGen & Agent Networks

The most powerful agentic systems use multiple specialized agents working together. Study **CrewAI**: define agents with roles, goals, and backstories; assign tasks; let a crew collaborate hierarchically or sequentially. Learn **Microsoft AutoGen**: conversational multi-agent patterns, group chats, code-execution agents. Understand **agent roles**: Planner, Executor, Critic, Summarizer — classic division of labor. Master **inter-agent communication protocols**: how agents pass structured messages vs free-form conversation. Study **supervisor patterns**: one LLM orchestrating a team of specialized sub-agents. Learn when multi-agent adds value vs. when a single agent loop is simpler and more reliable.

Multi-agent systems

CrewAI

AutoGen

Agent roles

Supervisor pattern

Message passing

Hierarchical agents

Swarm patterns

▶ [CrewAI Full Course - freeCodeCamp](#)▶ [AutoGen Tutorial - freeCodeCamp](#)▶ [Multi-Agent with LangGraph - LangChain](#)▶ [AI Agent Networks - freeCodeCamp](#)

Start with LangGraph, not CrewAI. CrewAI is great for demos. LangGraph is what production teams actually use. It gives you fine-grained control over state, branching, and cycles — essential when an agent needs to loop, backtrack, or take conditional paths. Learn LangGraph first; everything else becomes easier.

// FRAMEWORK COMPARISON — WHEN TO USE WHAT

FRAMEWORK	BEST FOR	COMPLEXITY	PRODUCTION-READY?
LangGraph	Stateful, cyclic, complex agent flows	Medium–High	Yes — used at scale
LangChain	RAG, chains, quick prototyping	Low–Medium	Yes with care

// EVALUATION, SAFETY & PRODUCTION

Evals, Guardrails & Deploying Agents at Scale

8

• Weeks 20–28

PHASE 08 • AGENT EVALUATION & TESTING

Evals — The Discipline That Separates Hobbyists from Engineers

You cannot ship agents without systematic evaluation. Most people skip this — it's why most agents fail in production. Build **unit evals**: test individual LLM calls with expected outputs. Build **end-to-end evals**: does the full agent task succeed? Learn **LLM-as-judge**: use GPT-4 to score agent outputs on criteria like accuracy, helpfulness, safety, and format adherence. Use **RAGAS** for RAG evaluation: faithfulness, answer relevancy, context precision, recall. Learn **Braintrust**, **LangSmith evals**, and **Weights & Biases Weave** for structured eval pipelines. Build a **regression test suite** — every time you change a prompt or model, run your evals. Agents without evals degrade silently.

Ship-or-fail gate

LLM-as-judge

RAGAS

Braintrust

LangSmith evals

W&B Weave

Unit evals

Regression tests

▶ Agent Evals with LangSmith - LangChain

▶ RAGAS RAG Evaluation - James Briggs

▶ LLM-as-Judge Pattern - Weights & Biases

▶ Agent Testing Strategies - freeCodeCamp

9

• Weeks 24–34

PHASE 09 • AGENT SAFETY & GUARDRAILS

Guardrails, Prompt Injection Defense & Responsible Agents

Autonomous agents can cause real-world harm — they can delete data, send emails, make purchases, execute code. Safety is not optional. Learn **input/output guardrails** with NeMo Guardrails and Guardrails AI: validate LLM inputs/outputs against policy rules. Study **prompt injection attacks**: malicious content in tool results or user inputs hijacking agent behavior — this is the SQL injection of the agent era. Learn **sandboxing**: never let agents execute code outside a contained environment (E2B, Docker). Implement **human-in-the-loop (HITL)** for high-stakes actions: confirm before sending emails, deleting files, or spending money. Study **principle of least privilege** for agent tool access. Learn **Constitutional AI** principles for alignment.

Non-negotiable for prod

NeMo Guardrails

Guardrails AI

Prompt injection

E2B sandboxing

HITL pattern

Least privilege

Constitutional AI

▶ NeMo Guardrails - NVIDIA

▶ Agent Safety Patterns - freeCodeCamp

▶ Prompt Injection Attacks - Simon Willison

▶ Human-in-the-Loop Agents - LangGraph

// AGENTIC DESIGN PATTERNS — THE CORE PATTERNS EVERY ENGINEER MUST KNOW

ReAct Loop

Reason → Act → Observe → Repeat. The fundamental agent loop. LLM reasons about what to do, executes a tool, observes the result, then reasons again. Used in nearly every production agent.

▶ ReAct Pattern - LangChain

Plan-and-Execute

Planner LLM breaks a complex goal into subtasks upfront. Executor agents complete each step. More reliable for long-horizon tasks than pure ReAct. Used in OpenAI's deep research.

▶ Plan & Execute - LangGraph

Reflection Pattern

Agent generates a draft, a critic agent reviews it, the generator revises. Iterative self-improvement. Dramatically improves

Subagent Delegation

Orchestrator agent routes subtasks to specialized agents (a coding agent, a research agent, a writing agent). Each expert

// DEPLOYMENT & SPECIALIZATION

Shipping Agents to Production & Specialization Tracks

10

• Weeks 26–36

PHASE 10 • PRODUCTION DEPLOYMENT

Serving Agents — FastAPI, Streaming, Queues & Monitoring

Building an agent that works in a notebook is 20% of the job. Deploy it: serve agents as **streaming REST APIs** with FastAPI — SSE (Server-Sent Events) for real-time token streaming to frontend clients. Use **task queues** (Celery + Redis, or BullMQ) for async long-running agent tasks that can't block an HTTP request. Learn **containerization** with Docker — package your agent + dependencies. Deploy on **Cloud Run, Railway, Fly.io** for managed serverless. Add **observability**: structured logging with Loguru, tracing with LangSmith/Langfuse, metrics with Prometheus. Monitor **latency, token usage, and cost per agent run**. Set up alerts for agent failures. Study **rate limiting and cost controls** — uncapped agents can burn \$1000s in minutes.

Production essentials

FastAPI + SSE streaming

Celery + Redis

Docker

Cloud Run / Railway

Langfuse

Token cost tracking

Rate limiting

▶ FastAPI Full Course - freeCodeCamp

▶ Docker Full Course - TechWorld with Nana

▶ Agent Monitoring - Langfuse

▶ Async Task Queues - DataTalks.Club

▶ Cloud Run Deployment - freeCodeCamp

11

• Month 9–14 (Specialization)

PHASE 11 • ADVANCED SPECIALIZATION

Computer-Use Agents, Fine-tuning & Frontier Systems

Once you're comfortable with the full stack, specialize. **Computer-Use Agents**: Anthropic's Computer Use API, browser agents with Playwright/Stagehand, desktop automation — agents that can actually operate a computer like a human. **Voice Agents**: realtime speech pipelines with Deepgram (STT) + LLM + ElevenLabs/Cartesia (TTS) — sub-300ms latency for real conversations. **Fine-tuning for agents**: fine-tune Llama 3.1 or Mistral on agent trajectories (tool use examples) to improve reliability and reduce costs vs. GPT-4. Study **MCP (Model Context Protocol)** — Anthropic's standard for agent-tool integration. Learn **OpenAI Assistants API** and **Responses API** for managed agent infrastructure. These are the skills unlocking senior and research roles.

Frontier skills

Computer Use API

Browser agents (Stagehand)

Voice agents (Deepgram)

Agent fine-tuning

MCP protocol

OpenAI Responses API

Realtime API

▶ Computer Use Agents - freeCodeCamp

▶ Voice Agents with Deepgram - Hugging Face

▶ Fine-tuning for Tool Use - Hugging Face

▶ MCP Protocol Deep Dive - LangChain

// SPECIALIZATION TRACKS — PICK YOUR PATH

TRACK	FOCUS	KEY TOOLS	WHO'S HIRING
Coding Agents	Code gen, review, debugging, repo navigation	Code Interpreter, Tree-sitter, E2B	Cursor, GitHub, Sourcegraph, startups
Browser / Web Agents	Web scraping, form-filling, research automation	Playwright, Stagehand, Computer Use	Automation cos., enterprises, SaaS
Voice Agents	Real-time conversational AI, call centers	Deepgram, ElevenLabs, LiveKit, VAD	Retell AI, VAPI, healthcare, sales

// SKILL MAP & PROJECTS

Full Timeline & Portfolio Projects to Build

■ MONTH 1-4

- Python async + Pydantic
- LLM fundamentals + APIs
- Prompt engineering (CoT, ReAct)
- Function calling / tool use
- RAG pipeline basics
- Vector DB (Qdrant / Pinecone)
- LangChain fundamentals

■ MONTH 5-9

- LangGraph stateful agents
- Multi-agent (CrewAI / AutoGen)
- Agent memory systems (Mem0)
- LangSmith tracing + evals
- RAGAS evaluation
- Guardrails + prompt injection
- FastAPI + SSE deployment

■ MONTH 10-18

- Computer-use / browser agents
- Voice agent pipelines
- Agent fine-tuning (LoRA)
- MCP protocol integration
- Cost optimization & caching
- Specialization track depth
- Published demos + blog

// PORTFOLIO PROJECTS — BUILD THESE 5 TO GET HIRED

 Deep Research Agent

Agent that takes a complex question, plans sub-questions, searches the web (Tavily), reads and synthesizes sources, writes a structured report with citations. Deployed as a web app with streaming output. Uses LangGraph + RAG + LangSmith evals.

[▶ RAG Pipeline - freeCodeCamp](#) **Coding Assistant Agent**

Agent that reads a GitHub repo, understands the codebase via RAG, accepts feature requests, writes code, executes it in an E2B sandbox, debugs failures, and opens a PR. The mini-Cursor. Shows you understand the full agentic coding loop.

[▶ E2B Code Execution - freeCodeCamp](#) **Personal AI Assistant**

Multi-tool agent with long-term memory (Mem0), calendar integration, email drafting, web search, and document Q&A over your own notes. Voice input via Whisper. Demonstrates tool use, memory, and integration engineering.

[▶ LangChain Agent Tools](#) **Multi-Agent Pipeline**

CrewAI or LangGraph system with 3+ specialized agents (Researcher → Writer → Editor → SEO Optimizer) collaborating on long-form content. Shows multi-agent orchestration, inter-agent messaging, and quality control loops.

[▶ CrewAI Full Course](#)

// DAILY ROUTINE

The Boring Agentic AI Routine That Works

- ☐ Read one agentic AI paper or blog post (Simon Willison, Lilian Weng, or Anthropic research blog)
- ☐ 1 hour of hands-on building — one new tool, one new eval, one deployment improvement
- ☐ Run your eval suite — catch regressions before they catch you in production
- ☐ Track one failure mode — what did your agent do wrong today? Write it down
- ☐ Share one build update, failure, or insight on LinkedIn or X/Twitter

// MASTER RESOURCE LIST

Best Free YouTube Channels for Agentic AI

 Andrej Karpathy

Former Tesla AI Director, OpenAI co-founder. "Neural Networks: Zero to Hero" is mandatory. His LLM talks explain exactly how the models your agents run on work internally. Watch everything he posts.

[▶ @AndrejKarpathy](#) **LangChain Official**

The primary channel for LangGraph, LangChain, and LangSmith tutorials. State-of-the-art agentic patterns from the team that built the most widely-used agent framework. Follow for new pattern releases.

[▶ @LangChain](#) **freeCodeCamp**

Full-length free courses on LangChain, CrewAI, AutoGen, FastAPI, Docker, and more. The best place for comprehensive, project-based agentic AI learning with zero cost.

[▶ @freecodecamp](#) **James Briggs**

The clearest technical tutorials on RAG, vector databases, semantic search, and agent memory. His Pinecone collaboration videos are the best resource for production-grade knowledge retrieval systems.

[▶ @jamesbriggs](#) **AI Jason**

Practical walkthroughs of agent patterns, RAG pipelines, and LlamaIndex workflows. Great for going from theory to working code. Projects are real-world and well-explained.

[▶ @AIJasonZ](#) **Krish Naik**

India's most practical AI educator. Deep dives on LangChain, LlamaIndex, Hugging Face, and agentic deployments. Best for bridging the gap between tutorials and real-world implementation.

[▶ @krishnaik06](#)

// TOOLS BY TBE – USE THESE

[DSA Yatra – Daily practice](#)[Prep Yatra – Interview tracker](#)[Tech Yatra – Learning roadmaps](#)[Resume Yatra – ATS-ready resume](#)[Shiksha – Free courses](#)[YouFocus – Distraction-free YT](#)[Interview Prep – Question banks](#)[Community – Peer learning](#)

The Agentic Era Is Here

The engineers building autonomous AI systems today are defining how software works for the next decade.
Start small. Build one tool-using agent. Deploy it. Then make it smarter.

[→ theboringeducation.com](#)

// FIND US EVERYWHERE



INSTAGRAM

[@theboringeducation](#)

LINKEDIN

[The Boring Education](#)

GITHUB

[The-Boring-Education](#)

